SSURJET

Sir Syed University Research Journal of Engineering & Technology
2024, Vol. 14, No. 2,
https://doi.org/10.33317/ssurj.653

A Deep Reinforcement Learning Framework for Detecting Fraudulent Bank Account Openings

Abdul Qayoom[1,2], Wu Yadong[1,3], Wang Song[1], Sammar Abbas[4], and Nadeem Ghafoor[1]

[1]*School of Computer Science and Technology, Southwest University of Science and Technology, Mianyang, China*
[2]*Lecturer, Department of Computer Science, Lasbela University of Agriculture, Water and Marine Sciences, Balochistan, Pakistan*
[3]*School of Computer Science and Engineering, Sichuan University of Science and Engineering, Zigong, Sichuan, China*
[4]*School of Information Engineering, Southwest University of Science and Technology, Mianyang, Sichuan, China*

Correspondence Author: Wu Yadong (wuyadong@suse.edu.cn)

Received October 07, 2024; Revised November 29, 2024; Accepted December 04, 2024

Abstract

Online banking has become more popular and widespread. A huge number of customers open new bank accounts every day, which leads to a rise in fraudulent account openings. Many fraudulent activities related to this have been reported over time. It has remained a difficult task to detect fraudulent accounts efficiently. Artificial Intelligence (AI) is performing well in all the domains in the real world. So, the objective of this research is to build the most effective model to detect electronic financial transaction fraud in bank account opening. This research work aims to solve this problem (1) how to use deep reinforcement learning (DRL) for the detection of fraudulent bank accounts and (2) develop a Deep Q-Networks (DQN) based solution architecture for the detection of bank account opening fraud in an efficient way. Our proposed model achieved the highest fraud detection accuracy with 97% using the DQN algorithm with the benchmark Kaggle dataset for bank account fraud detection.

Index Terms: *Artificial Intelligence in Banking, Bank Account Fraud Detection, Deep Reinforcement Learning, Deep Q-Networks, Financial Fraud Detection.*

I. INTRODUCTION

The detection of fraud has become increasingly important in the present financial environment, especially as online banking becomes more widespread, and more people open accounts there. Financial service providers are confronted with obstacles from dishonest individuals who take advantage of weaknesses in the account opening procedures, resulting in substantial monetary losses and harm to their image. Strong real-time detection systems for fraud are needed as banks digitalize. As fraud strategies evolve, rule-based fraud detection methods become less effective. Traditional systems that use defined rules and patterns to detect suspicious activity typically produce many false positives and are sluggish to adapt to complex fraud techniques. Machine Learning (ML) makes fraud detection more flexible and adaptable. ML algorithms can analyze large datasets, find difficult patterns, and improve over time. These algorithms may better identify suspicious account creation attempts by reviewing previous records of fraud and appropriate actions. ML also improves fraud detection.

Banks are under pressure to combat bogus account openings. A recent study shows that ML approaches prevent fraud. One research used 'Game Theory' and 'Neural Networks' to identify fraud with a 91.7% True Positive Rate (TPR) and 0.0% False Positive Rate (FPR) [1]. Another research used a 'Stacking' ensemble-based model to detect fraudulent transactions with 98% accuracy, demonstrating the relevance of precision and recall in fraud detection [2]. With a fraud detection rate of 95.67%, the XGBoost algorithm is an effective tool for real-time suspicious activity identification [3]. These studies highlight that advanced ML algorithms have the power to improve security and can combat money laundering and identity theft [4], and [5].

The Naive Bayesian Network, developed using ML, can detect fraudulent bank account openings and improve detection and prevention of fraud [6]. Fang, Lv, et al. [7] used convolutional neural networks using heterogeneous data to identify financial fraud in novel methods.

Intelligent models powered by artificial intelligence are being developed to better evaluate huge volumes of data and improve fraud detection systems. Deep learning models have performed well in several areas, strengthening their significance in financial security. Notable areas where deep learning has proven effective include the analysis of medical images, object recognition, natural language processing, genome structural analysis in Bioinformatics, and the detection of fraudulent activities. Decisions made using machine learning methods are typically based on the known features of the training data. The reinforcement learning algorithm optimizes a criterion, which consists of loss, reward, and penalty, for both the training and test data [8].

Deep Q-Networks (DQN) excels in detecting complicated fraud patterns. The algorithm is trained on one million real-world instances to detect account opening fraud. DQN uses

reinforcement learning to learn and adapt, unlike conventional machine learning approaches. This adaptability makes the fraud detection system less vulnerable and responsive to new threats.

The main objective of this study has been defined below:

- This research aims to reduce bank account opening fraud while improving financial security.
- The research improved fraud detection models using DQN.
- Finally, this study compares the results with existing research works to highlight the improvements to identify fraudulent bank account openings using deep reinforcement learning.

Furthermore, the structure of the paper is as; Literature review presents the existing research on bank account opening fraud detection. Proposed methodology describes this study's approach and examines the dataset's characteristics. Results and discussion evaluates the model's performance and conclusion section summarizes the findings and provides a final analysis.

II. LITERATURE REVIEW

With advancements in financial systems, fraud chances have grown as well, making new financial crime detection and prevention measures necessary. Machine Learning (ML) techniques may solve Bank Account opening Fraud (BAF). This literature review discusses new research on ML approaches, their strengths and weaknesses, and the relevance of class imbalance and fairness in fraud detection. This section mainly concerns in investigating the efficacy of machine learning algorithms and deep learning algorithms. However, there are few research gaps and limitations in this area. Evaluation metrics indicate low model performance while lack of feature selection procedure prevents replicability of the results. The proposed approaches are not effective and too computationally expensive for real-world use.

Data analytics helps modern banks handle huge amounts of data every day to make instructed choices, improve services, and maximize profits [9]. Bank account fraud continues to plague organizations and clients. Bank account fraud happens when unauthorized parties abuse bank account information, causing cash losses, credit damage, and financial institution reputation damage [10], and [11].

Modern fraudsters utilize advanced strategies that traditional fraud protection systems cannot handle. Machine learning may uncover fraud patterns and anomalies, offering possible measures. Data quality, model interpretability, and keeping up with fraud's continual evolution make fraud detection difficult [12], and [13]. Therefore, understanding the various types of bank account fraud and overcoming these obstacles is essential to maintaining the banking system's integrity when using machine learning for fraud detection.

Fraud detection of credit cards has attracted many researchers in the last few years. For example, numerous studies focused on credit card fraud problems have used machine learning techniques. The research techniques found in the current literature have mostly used machine learning algorithms, both supervised as well as unsupervised methods. In supervised learning, the frequently used techniques include; Random Forest, Support Vector Machine (SVM), Artificial Neural Networks, Long Short Term Memory (LSTM), Logistic Regression models, Naive Bayes, Decision Trees, and Ensemble techniques [14-16]. Also, both classical machine learning methods and various deep learning techniques including; Convolutional Neural Networks (CNN), Long Short Term Memory (LSTM), Gated Recurrent Unit (GRU), and Deep Q-Networks (DQN) have been addressed [17-19].

Others have employed unsupervised learning methods such as; Auto-Encoders, Isolation Forest and Clustering for credit card fraud detection. These techniques are useful in addressing the problem of unlabeled data. Some attempts have been made to explore semi-supervised models which solve the limitation above by making use of both labeled and unclearly labeled data in order to boost model performance [20], and [21].

A comparison of the test results of our model with existing models and methods is given below in Table I.

Table I: Comparison of the DQN Model with Existing Models

S. No.	Year	Method/ Model	Dataset	Accuracy (%)	Reference
1.	2019	Random Forest	Bank Account Dataset	94.7	[5]
2.	2020	Support Vector Machine (SVM)	Bank Account Dataset	94	[5]
3.	2024	KNN	Synthetic Bank Dataset	91.67	[30]
4.	2024	DQN Model (Proposed)	Kaggle Bank Account Fraud (BAF) Dataset	97	Our Proposed Work

III. METHODOLOGY

This section defines the overall strategy of our work, in the first subsection we define the dataset like the dataset used in the bank account fraud detection system consists of client records that include essential features such as transaction amount, date, time, location, customer information, and an indicator of whether the account was fraudulent. In the second subsection, before applying the dataset to our model, the raw data undergoes preprocessing to ensure it is clean and suitable for analysis. This process involves handling missing or incomplete data, encoding categorical variables, normalizing or standardizing numerical features, and balancing the dataset to address class imbalance. Additionally, feature selection is performed to identify the most relevant variables that contribute to detecting fraudulent activities, which improves the model's performance. Further, the choice of model could range from traditional models to more advanced techniques but we selected a deep reinforcement model and trained it to detect fraudulent account openings as shown in Figure 1.

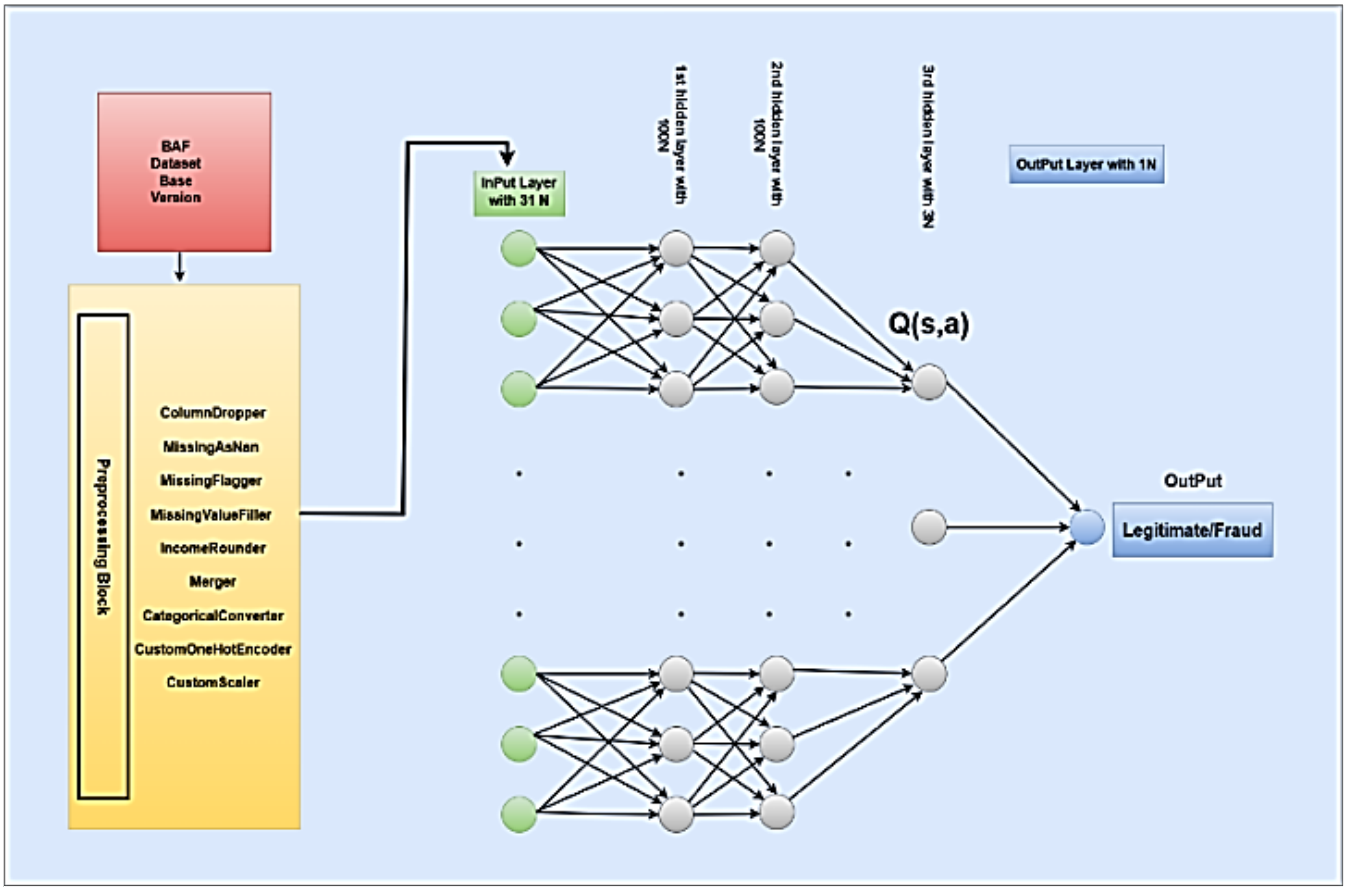

Figure 1: System Architecture of BAF

A. Dataset

The original data suite contains six data variations, including differences in outcome prevalence, group size, and feature separability, which were used to replicate controlled bias. Especially in static and time series environments, biases are associated with protected attributes such as; customer_age, income, and employment_status, which provide an opportunity to fully assess the accuracy and performance of the model. The study of performance problems is facilitated by controlled biases, especially when examining prevalence, group size, and distributional disparities.

We utilize the base version of the Bank Account Fraud (BAF) dataset in our work, which is a valuable resource that Jesús et al. published in 2022 [22]. The BAF dataset is significant because it prioritizes algorithmic fairness and provides a comprehensive tabular dataset suitable for machine learning evaluations. More than one million entities with 31 features and eight months of temporal data are included in the BAF dataset. We split the dataset i.e., 80% for training and 20% for testing with random_state. A synthetic data pipeline based on anonymous bank fraud detection data in the real world was used to build this dataset. It describes real-world issues related to fraud detection, including data bias, temporal dynamics, and class imbalance.

Additionally, a Generative Adversarial Network (GAN) [23] focused on tabular data, was used to synthesize the data by utilizing CTGAN [24]. Sensitive attributes, such as the customer_age and income, are truncated and categorized to protect privacy. Strict precautions are taken to ensure that duplicate occurrences are not included. The data is mainly divided into fraudulent (positive class) and legitimate (negative class) applications, with a noticeable class imbalance, with the prevalence of fraud ranging from 0.85% to 1.5% in eight months.

The dataset includes several variations of datasets with the aim of comparing deep learning and machine learning models for static and dynamic parameters while controlling for biases. However, because it lacks a number of privacy requirements and may contain potential errors, it is not intended for direct use in production fraud detection models. Notwithstanding these drawbacks, this resource simulates real-world application challenges and provides a useful framework for the performance evaluations of current deep learning and machine learning models.

B. Data Analysis and Processing

The data used in this study comprises one million occurrences with 31 features frequently used in fraud detection models. A crucial column called "month" presents temporal data, indicating the exact time each attribute appeared. This allows us to analyze trends over time. A comprehensive basis for the creation and evaluation of fraud detection models is provided by the dataset, which also contains protected attributes such as; customer_age, employment_status and income percentage [22].

Among the 31 features, 26 are numerical, and 5 are categorical. In which 12 discrete variables and 14 continuous variables constitute the numerical attributes. The distribution of income levels of the customer is not

uniform. The highest income is those in the 0.9 percentile of income, followed by those in the 0.1 and 0.8 brackets. Although the income distribution looks abnormal and not significantly skewed, the data in the customer_age distribution column show a bias to the left, suggesting that most applicants are in the age range from 10 to 40 years. 30 years is the average age, with half of the candidates in the age group of 20-40.

The dataset shows a noticeable imbalance between classes, which is a common challenge associated with the fraud detection domain when classifying account applications into fraudulent and non-fraudulent categories. The percentage of fraudulent cases in the entire dataset is about 1%. Given this imbalance, it is essential to apply sampling techniques to ensure that machine learning models are trained efficiently in both classes. This approach will help to mitigate the issues posed by the unbalanced nature of the data and provide a fair, accurate, and more balanced model.

In real-world datasets, the data are often incomplete, inconsistent, or contain features unsuitable for direct processing by machine learning algorithms. Therefore, preprocessing is an essential step to transform raw data into a standardized and clean format, thereby ensuring robust and accurate model training. In this research, preprocessing was performed on the BAF dataset, with key functions including; encoding categorical variables, handling missing values, and scaling functions.

The first stage in preprocessing involved addressing missing values, a common issue in real-world data. Missing values can arise either randomly or follow discernible patterns based on the data itself. The approach to handling these values depends on the underlying reasons and their distribution patterns. In this dataset, several features such as previous address month's count, intended balance amount, and bank month's count exhibited a substantial number of missing values. While these missing values could significantly affect analysis, the features still demonstrated correlations with the target feature (fraud/non-fraud). Notably, negative values were used to denote missing entries, a choice that could preserve useful information. The implications of transforming these negative values into explicit missing indicators remain to be studied, as this may risk potential information loss.

For data preprocessing, we designed classes to be modular and reusable, allowing them to fit into a 'scikit-learn' pipeline for preprocessing and feature transformation in model workflows.

Each class handles a specific data preprocessing task, ensuring that the dataset is clean, consistent, and ready for model training:

- ColumnDropper Class: This class is designed to drop specified columns from a dataset. It inherits from BaseEstimator and TransformerMixin, making it compatible with scikit-learn pipelines.
- MissingAsNan Class: This class replaces specific missing or negative values with NaN in the dataset. It is useful for marking missing or invalid entries before further processing.
- MissingFlagger Class: This class flags missing values in specified columns by creating binary indicators. If a value is missing, the indicator will be 1, otherwise 0.
- MissingValueFiller Class: This class handles missing values by filling them with a specified value (default is 0).
- IncomeRounder Class: This class rounds the income column values to one decimal place.
- Merger Class: This class merges specific categories in categorical columns to simplify the dataset.
- CategoricalConverter Class: This class converts the data type of specified categorical columns into category type, which is useful for efficient storage and manipulation.
- CustomOneHotEncoder Class: This class performs one-hot encoding on categorical features while maintaining a data frame format for easy integration with scikit-learn pipelines.
- CustomScaler Class: This class standardizes numerical columns by scaling them, making data easier to work with for algorithms sensitive to the magnitude of features.

The next preprocessing step involved encoding categorical variables into a numeric format suitable for the model. Since algorithms cannot process categorical features such as housing_status or employment_status, one-hot encoding was applied using the Pandas library. This method converted categorical variables into binary indicator columns, where each column represents one of the categories.

Finally, feature scaling was used to ensure that all features had a comparable range of values. This was achieved using StandardScaler of Scikit-learn, which standardized the features by scaling and then normalized the feature values, ensuring they were comparable by changing each feature to have a mean of 0 and a standard deviation of 1. Normalization ensures that all numerical features are of comparable size, preventing specific features from dominating the learning process due to their larger scale.

This transformation was performed using the following equation i.e., Eq. 1.

$$z = \frac{x-\mu}{s} \quad (1)$$

Where:
z represents the scaled value,
x is the original feature value,
μ is the mean of the feature,
s is the standard deviation of the feature.

The following algorithm (Figure 2) shows the feature scaling and normalization as the final step in data preprocessing.

All the features were scaled uniformly by utilizing this normalization strategy which prevents any single feature from dominating the model due to its wider numerical range. This procedure is essential in machine learning tasks, especially when features vary significantly in the value range because it ensures that each feature contributes equally to the model learning process [25]. Without proper

scaling, features that span wider ranges can have a disproportionate impact on model performance, resulting in biased predictions.

Input: Dataset D with features X = {x_1, x_2, ..., x_n}
Output: Preprocessed dataset D_scaled
1. Set scaling method (e.g., Standardization or Min-Max Scaling)
2. Set normalization method (e.g., L2 normalization)
3. **For each feature x_i in X:**
   a. If using Standardization:
      i. Calculate the mean (μ) and standard deviation (σ)
      ii. Scale feature: $x_i' = (x_i - \mu) / \sigma$
   b. If using Min-Max Scaling:
      i. Find min(x) and max(x)
      ii. Scale feature: $x_i' = (x_i - \min(x)) / (\max(x) - \min(x))$
4. **For each sample in D:**
   a. If using L2 normalization:
      i. Calculate norm = sqrt(sum(xi^2))
      ii. Normalize sample: x_normalized = x / norm
5. Handle missing values:
   a. Fill missing values using mean, median, or k-NN imputation
6. **Return** D_scaled

Figure 2: Algorithm Pseudocode for Standardization and Normalization of Dataset

C. Model Selection and Deployment

In this study, we use Reinforcement Learning (RL) to create an end-to-end agent capable of performing early categorization with high efficiency. The fundamental contribution of this study is the creation of an early classifier agent based on the presentation of a well-defined set of states and actions for reinforcement learning. In addition, we offer a customized system of rewards that aims to balance the trade-off between attentiveness and accurate classification [26].

Compared to traditional algorithms that do not distinguish between behavior and learning, Q-learning uses an off-policy approach, effectively separating the policy of action from the policy of learning [27]. This distinction ensures that information from a suboptimal action in the next state is not used to update the Q function of the current state, solving the problem that previous approaches involved incorrect decisions in learning updates. Q-learning, by using its off-policy mechanism, resolves this dilemma.

The Q-value updated formula is defined in Eq. 2 [28].

$$Q(s,a) \leftarrow Q(s,a) + \alpha[R + \gamma \max Q(\acute{s},\acute{a}) - Q(s,a)] \quad (2)$$

In this equation:
α represents the learning rate,
a value between 0 and 1,
while R denotes the reward, with its value diminishing over time.

The Q-value Q(s,a) for the current state-action pair is updated by incorporating the best possible action for the next state, thus continuously refining the Q-value at each state using the aforementioned equation. At the start of Q-learning, rewards are pre-populated in the Q-table. As an agent selects an action based on a policy from an initial state, it transitions to the next state according to the updating rule [27]. This iterative process continues until the Q-values converge, resulting in the Q-table being used to solve the problem at hand.

Q learning combines dynamic programming with Monte Carlo methods, which are commonly used to solve Bellman equations. This hybrid technique has become the basis of many reinforcement learning algorithms due to its simplicity and effectiveness in a single-agent environment. In contrast to other methods, Q-learning has extensive learning capabilities, although with a limitation: updates are performed only once per action. This scenario limits the capability of the algorithm to deal with complicated challenges in vast action-state spaces because many action-state pairs cannot be visited. Furthermore, Q learning necessitates extensive memory storage because the Q table for rewards must be initialized in advance. In multi-agent situations, where many agents interact, this memory requirement becomes even more troublesome as the action-state space increases significantly. Therefore, the underlying Q-learning algorithm tries to perform efficiently in multi-agent scenarios and highlights its limitations in these situations.

Google DeepMind launched Deep Q-learning, which combines Convolutional Neural Networks (CNN) with the original Q-learning method. The traditional Q-learning method struggles to express the value function for each state in high-dimensional state space environments [27]. To address this issue, Deep Q-learning uses CNNs as feature approximators, allowing accurate estimation of Q values in complex contexts. In addition to using CNN for value approximation, deep Q-learning uses two essential techniques; target Q and experience replay approach. The combination of these methods helps to face the instability that is sometimes associated with the approximation of the value based on the neural network. Particularly, experience replay plays an important role in stabilizing the learning process by storing agents' experiences and applying them in batches to subsequent training samples, thus improving the convergence and robustness of the model.

The training of the early classifier agent is conducted using the standard Deep-Q-Network (DQN) algorithm. DQN aims to approximate the agent's behavior function by utilizing a deep neural network in conjunction with Q-learning. This RL approach has been widely used in sequential decision-making tasks, such as training artificial intelligence to play video games [29]. In our study, the agent interacts with its environment through episodes, where it observes its current state, selects an action and receives feedback in the form of a reward.

The agent's objective is to maximize the total discounted reward over time as described in equation i.e., Eq. 3 [29].

$$g_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k} \quad (3)$$

Where;
γ is the discount factor controlling the weight of future rewards,

$\gamma \in [0,1]$ where if $\gamma = 0$: the agent is short-sighted, considering only immediate rewards, and if γ is close to 1: the agent values future rewards more, effectively considering long-term gains.
The agent's policy, $(\pi\, a/O)$, models the probability of selecting action (a) given a state (O), aiming to discover the optimal policy (π *).

Through interactions with the environment, the agent iteratively refines its behavior by storing transitions (O_t, a_t, r_t, $O_{\{t+1\}}$) in replay memory and applying gradient descent on the loss function in the below equation, i.e., Eq. 4 derived from the DQN [28].

$$L_t(\theta)_t = (r + \gamma \max_{\acute{a}} Q(\acute{O}, \acute{a}, \theta_t^-) - Q(O, a, \theta_t))^2 \quad (4)$$

Here;
θ_t represents the DQN's parameters, and
θ_t^- is a fixed, older version of the network used to stabilize training. Over time, the agent updates its Q-function and learns the relationship between actions, states, and future rewards, gradually improving its decision-making capabilities.

We defined the environment class in the model, which takes processed data as input and returns rewards based on the Z-score of previous observations. If the Z-score exceeds 3, the account is considered abnormal, and a negative reward is issued. The environment keeps track of past observations (history) and iterates through the data, producing observations, rewards, and done flags. Next, we implemented the neural network used in the DQN algorithm. The architecture consists of the input layer, 3 fully connected hidden layers (fc1, fc2, fc3), and the output layer, which process the input data. Layer normalization (ln1, ln2) is applied after the first and second fully connected layers to stabilize learning. The ReLU activation function is applied after the first and second layers. The network outputs Q-values for different actions.
In training, the train_dqn function trains the DQN using an experience replay buffer to store past experiences. It applies the Bellman equation to update the Q-values and uses the Adam optimizer for weight updates. Key techniques include epsilon-greedy exploration, memory replay, and periodically updating the target network.

The training process follows these steps:

- Collects experiences in the environment.
- Stores them in memory for future mini-batch updates.
- Every few steps, samples batches from memory and computes the target Q-values using the Bellman equation.
- Updates the network weights using the computed loss.
- Periodically updates the target network with the current Q-Network weights.
- It also decays the exploration parameter and calculates performance metrics like accuracy and recall.

For validation, the valid_dqn function validates the performance of the trained model by calculating loss, and accuracy on a separate validation environment.

IV. RESULTS AND DISCUSSION

For the simulations of the proposed model, a maximum of 10 epochs was established. The Q-network model utilized the Adam optimizer for gradient-based optimization. Each epoch iteration contained 100 sub-epochs, with a total maximum of 500 steps. The memory buffer size was set to 250, and a batch size of 20 was used for training. The exploration parameter epsilon was initialized at 1.0, with an epsilon decay rate of 0.0003, and a minimum epsilon (ε) value fixed at 0.1. Where $\epsilon = \max(\epsilon min, \epsilon decay \times \epsilon)$, ϵmin is the minimum threshold for exploration, and $\epsilon decay$ controls how fast ε decreases over time.
The training frequency was set to 10 steps, while the Q-network model was updated every 20 steps. The discount factor, gamma, was set to 0.97. The experiments were conducted using 'Google Colab', which provided essential computational resources, including 12 GB of RAM and a T4 GPU, significantly improving the efficiency of the simulations and accelerating the model's training process.
To evaluate the performance of the proposed model, a deep neural network was assessed by examining its accuracy and loss during both the training and validation phases. As illustrated in Figure 3, accuracy showed a consistent improvement throughout the training and validation process, with observable trend lines indicating the model's learning progression.

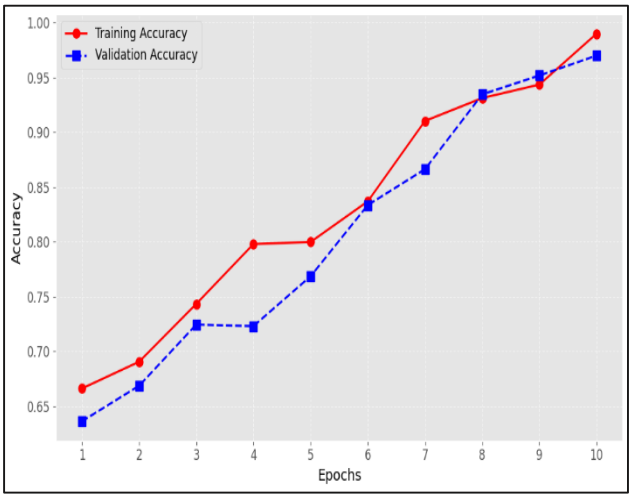

Figure 3: Training and Validation Accuracy on BAF dataset per Epoch [30]

Similarly, Table II presents the training and validation accuracy and loss for each epoch. Figure 4 demonstrates that the loss steadily decreased during both training and validation, indicating a reduction in prediction errors over time. The proposed model achieved the training accuracy of 99% and validation accuracy of 97% while the training loss reduced to 0.20 while the validation loss is 0.28. These results suggest that the proposed bank account fraud detection model exhibits robust performance when compared to other classical or base-line algorithms, as discussed in the literature review.

Table II: Training and Validation Accuracy and Loss per Epoch

Epoch	Training Accuracy	Validation Accuracy	Training Loss	Validation Loss
1	0.6661	0.6362	0.4606	0.5423
2	0.6895	0.6686	0.4777	0.5102
3	0.7433	0.7245	0.4291	0.5057
4	0.7979	0.7231	0.4003	0.4581
5	0.7898	0.7686	0.3537	0.4187
6	0.8329	0.8327	0.3309	0.4085
7	0.9103	0.8662	0.3057	0.3591
8	0.9245	0.9315	0.2614	0.3444
9	0.9494	0.9517	0.2450	0.2937
10	0.9899	0.9699	0.2096	0.2843

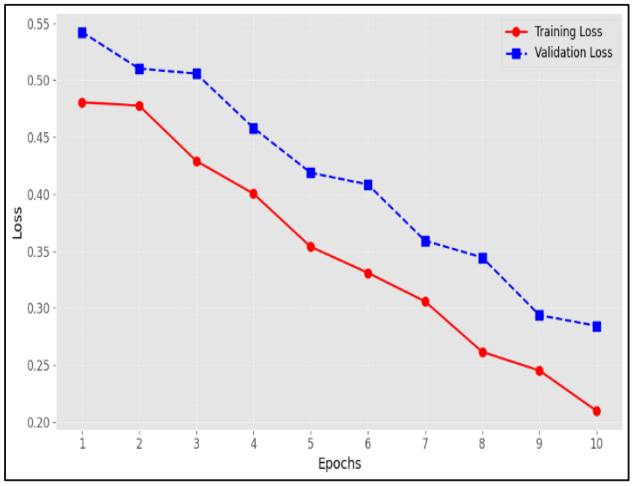

Figure 4: Training and Validation Loss on BAF dataset per Epoch

V. CONCLUSION

In this work, a novel technique for fraudulent bank account openings was proposed using a deep reinforcement learning scheme. The system was implemented using a benchmark dataset for bank account opening fraud detection through a deep Q networks algorithm. The proposed model was able to achieve 97% accuracy which is the highest accuracy so far by any model implementation. The proposed model shows better performance when compared to other traditional or base-line algorithms. Consequently, the model is well-suited for integration into core banking solutions to enhance fraud detection capabilities in fraudulent bank account openings.

Acknowledgment

The authors would like to thank the management of Lasbela University of Agriculture, Water and Marine Sciences, Balochistan, Pakistan, Southwest University of Science and Technology, Mianyang, China, and Sichuan University of Science and Engineering, Zigong, Sichuan, China, for their support and assistance throughout this study.

Authors Contributions

All the authors equally contributed to this research study.

Conflict of Interest

The authors declare no conflict of interest and confirm that this work is original and not plagiarized from any other source, i.e., electronic or print media. The information obtained from all of the sources is properly recognized and cited below.

Data Availability Statement

The testing data is available in this paper.

Funding

This research received no external funding.

References

[1] USMAN, AU, S. B. Abdullahi, J. Ran, Y. Liping, A. A. Suleiman, H. Daud et al.(2024). Modeling the Dynamic Behaviors of Bank Account Fraudsters using combined Simultaneous Game Theory with Neural Networks.

[2] Alhashmi, A. A., Alashjaee, A. M., Darem, A. A., Alanazi, A. F., & Effghi, R. (2023). An Ensemble-Based Fraud Detection Model for Financial Transaction Cyber Threat Classification and Countermeasures. *Engineering, Technology & Applied Science Research*, *13*(6), 12433-12439.

[3] Prayoga, A. H., & Voutama, A. (2024). Pengembangan Aplikasi Bank Account Fraud Detection Dengan Menggunakan Algoritma XGBOOST. *JATI (Jurnal Mahasiswa Teknik Informatika)*, *8*(3), 2916-2922.

[4] Kalyani, G., Mishra, A. K., Harish, D., Tyagi, A. K., Sajidha, S. A., & Pandey, S. (2024). Detection of Bank Fraud Using Machine Learning Techniques. *Automated Secure Computing for Next-Generation Systems*, 223-241.

[5] Desrousseaux, R., Bernard, G., & Mariage, J. J. (2019, September). Identify Theft Detection on e-Banking Account Opening. In *IJCCI* (pp. 556-563).

[6] Wan, W., & Li, W. (2021, March). A False Bank Accounts Detection Method Based on Naive Bayesian Network. In *Journal of Physics: Conference Series* (Vol. 1820, No. 1, p. 012094). IOP Publishing.

[7] Lv, F., Wang, W., Wei, Y., Sun, Y., Huang, J., & Wang, B. (2019). Detecting Fraudulent Bank Account Based on Convolutional Neural Network with Heterogeneous Data. *Mathematical Problems in Engineering*, *2019*(1), 3759607.

[8] Singh, V., Chen, S. S., Singhania, M., Nanavati, B., & Gupta, A. (2022). How are Reinforcement Learning and Deep Learning Algorithms used for Big Data Based Decision Making in Financial Industries–A Review and Research Agenda. *International Journal of Information Management Data Insights*, *2*(2), 100094.

[9] Nugroho, A. S. E., & Hamsal, M. (2021). Research Trend of Digital Innovation in Banking: a Bibliometric Analysis. *Journal of Governance Risk Management Compliance and Sustainability*, *1*(2), 61-73.

[10] Yu, H., & He, X. (2021). Corporate Data Sharing, Leakage, and Supervision Mechanism Research. *Sustainability*, *13*(2), 931.

[11] Kemp, S., & Erades Pérez, N. (2023). Consumer Fraud Against Older Adults in Digital Society: Examining Victimization and its Impact. *International Journal of Environmental Research and Public Health*, *20*(7), 5404.

[12] Žigienė, G., Rybakovas, E., & Alzbutas, R. (2019). Artificial Intelligence Based Commercial Risk Management Framework for SMEs. *Sustainability*, *11*(16), 4501.

[13] Amin, A., Rahmawaty, M. F. L., Masrom, S., & Rahman, R. A. (2023). Tree-Based Machine Learning and Deep Learning in Predicting Investor Intention to Public Private Partnership. *en. In: International Journal of Advanced Computer Science and Applications*, *14*.

[14] Randhawa, K., Loo, C. K., Seera, M., Lim, C. P., & Nandi, A. K. (2018). Credit Card Fraud Detection using Adaboost and Majority Voting. *IEEE access*, *6*, 14277-14284.

[15] Awoyemi, J. O., Adetunmbi, A. O., & Oluwadare, S. A. (2017, October). Credit Card Fraud Detection using Machine Learning Techniques: A Comparative Analysis. In *2017 international conference on computing networking and informatics (ICCNI)* (pp. 1-9). IEEE.

[16] Dal Pozzolo, A., Boracchi, G., Caelen, O., Alippi, C., & Bontempi, G. (2017). Credit Card Fraud Detection: A Realistic Modeling and a Novel Learning Strategy. *IEEE Transactions on Neural Networks and Learning Systems, 29*(8), 3784-3797.

[17] Varmedja, D., Karanovic, M., Sladojevic, S., Arsenovic, M., & Anderla, A. (2019, March). Credit Card Fraud Detection-Machine Learning Methods. In *2019 18th International Symposium INFOTEH-JAHORINA (INFOTEH)* (pp. 1-5). IEEE.

[18] Roy, A., Sun, J., Mahoney, R., Alonzi, L., Adams, S., & Beling, P. (2018, April). Deep Learning Detecting Fraud in Credit Card Transactions. In *2018 Systems and Information Engineering Design Symposium (SIEDS)* (pp. 129-134). IEEE.

[19] Mahajan, S., Kolhar, S., Patil, A., Mahajan, S., Menpara, J., & Mahajan, A. (2024). Credit Card Fraud Detection using Reinforcement Learning. *International Journal of Computing and Digital Systems, 16*(1), 189-198.

[20] Sisodia, D. S., Reddy, N. K., & Bhandari, S. (2017, September). Performance Evaluation of Class Balancing Techniques for Credit Card Fraud Detection. In *2017 IEEE International Conference on Power, Control, Signals and Instrumentation Engineering (ICPCSI)* (pp. 2747-2752). IEEE.

[21] Melo-Acosta, G. E., Duitama-Munoz, F., & Arias-Londono, J. D. (2017, August). Fraud Detection in Big Data using Supervised and Semi-Supervised Learning Techniques. In *2017 IEEE Colombian Conference on Communications and Computing (COLCOM)* (pp. 1-6). IEEE.

[22] Jesus, S., Pombal, J., Alves, D., Cruz, A., Saleiro, P., Ribeiro, R. et al. (2022). Turning The Tables: Biased, Imbalanced, Dynamic Tabular Datasets For Ml Evaluation. *Advances in Neural Information Processing Systems, 35*, 33563-33575.

[23] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair,et al. (2014). Generative Adversarial Nets. *Advances in Neural Information Processing Systems, 27*.

[24] Xu, L., Skoularidou, M., Cuesta-Infante, A., & Veeramachaneni, K. (2019). Modeling Tabular Data using Conditional Gan. *Advances in Neural Information Processing Systems, 32*.

[25] Gebretsadik, A., Kumar, R., Fissha, Y., Kide, Y., Okada, N., Ikeda, H. et al (2024). Enhancing Rock Fragmentation Assessment in Mine Blasting Through Machine Learning Algorithms: A Practical Approach. *Discover Applied Sciences, 6*(5), 223.

[26] Martinez, C., Perrin, G., Ramasso, E., & Rombaut, M. (2018, September). A Deep Reinforcement Learning Approach For Early Classification of Time Series. In *2018 26th European Signal Processing Conference (EUSIPCO)* (pp. 2030-2034). IEEE.

[27] Jang, B., Kim, M., Harerimana, G., & Kim, J. W. (2019). Q-Learning Algorithms: A Comprehensive Classification and Applications. *IEEE Access, 7*, 133653-133667.

[28] Wang, X., Zhao, C., Huang, T., Chakrabarti, P., & Kurths, J. (2023). Cooperative Learning of Multi-Agent Systems Via Reinforcement Learning. *IEEE Transactions on Signal and Information Processing over Networks, 9*, 13-23.

[29] Zolfpour-Arokhlo, M., Selamat, A., Hashim, S. Z. M., & Afkhami, H. (2014). Modeling of Route Planning System based on Q Value-based Dynamic Programming with Multi-Agent Reinforcement Learning Algorithms. *Engineering Applications of Artificial Intelligence, 29*, 163-177.

[30] Usman, A. U., Abdullahi, S. B., Liping, Y., Alghofaily, B., Almasoud, A. S., & Rehman, A. (2024). Financial Fraud Detection using Value-at-Risk with Machine Learning in Skewed Data. *IEEE Access.*